Pratham Bhoot

Capstone Assignment-1: Report

Creation of Storage Account:

Home > Storage accounts >

Create a storage account ...

manage your storage account together with other resources.

Subscription * : LabsKraft

Resource group * : LabsKraft2027

Create new

Instance details

Storage account name * : prathamstorage1

Region * : (US) West Central US

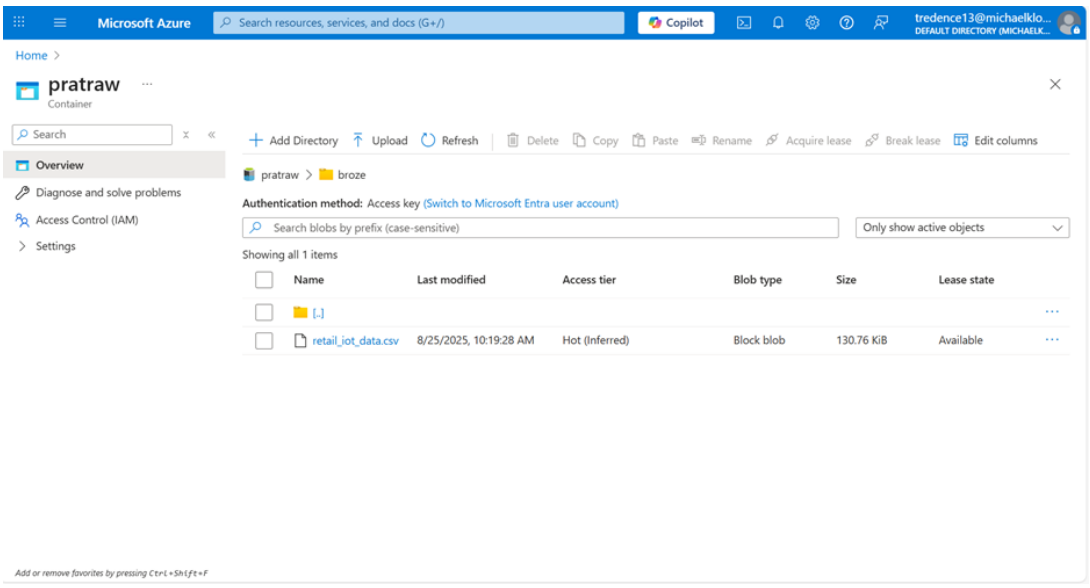
Deploy to an Azure Extended Zone

Primary service : Azure Blob Storage or Azure Data Lake Storage Gen 2

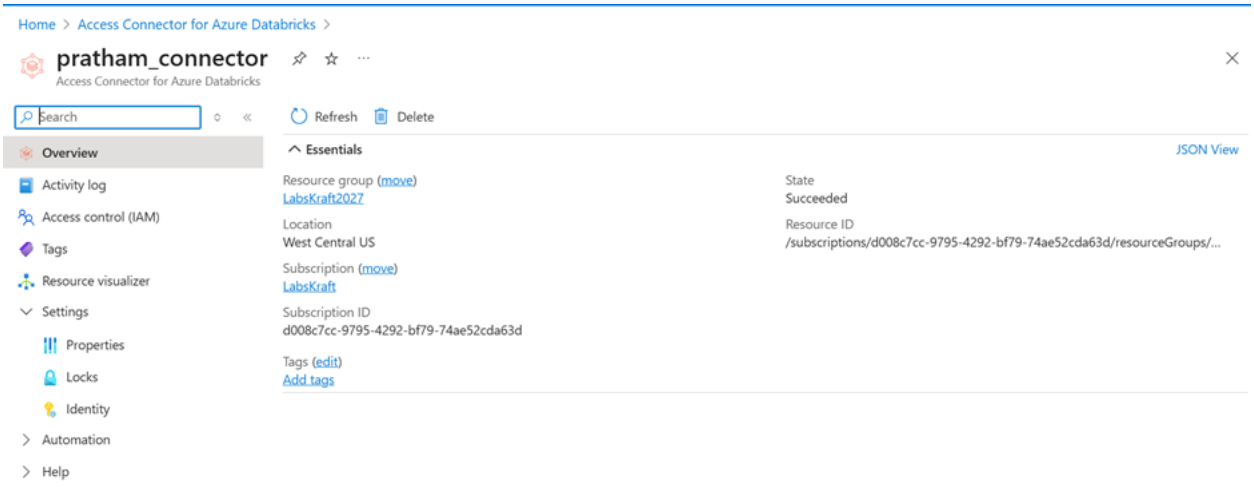
Performance * : **Standard:** Recommended for most scenarios (general-purpose v2 account)

Premium: Recommended for scenarios that require low latency.

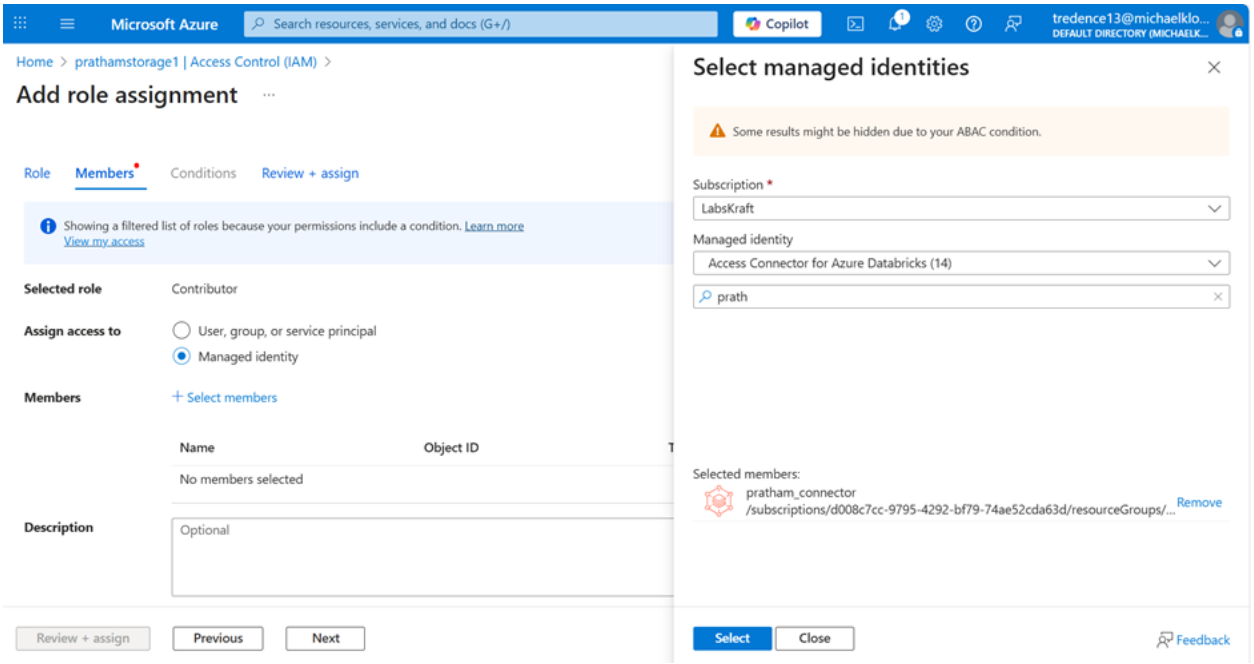
Redundancy * : Locally-redundant storage (LRS)



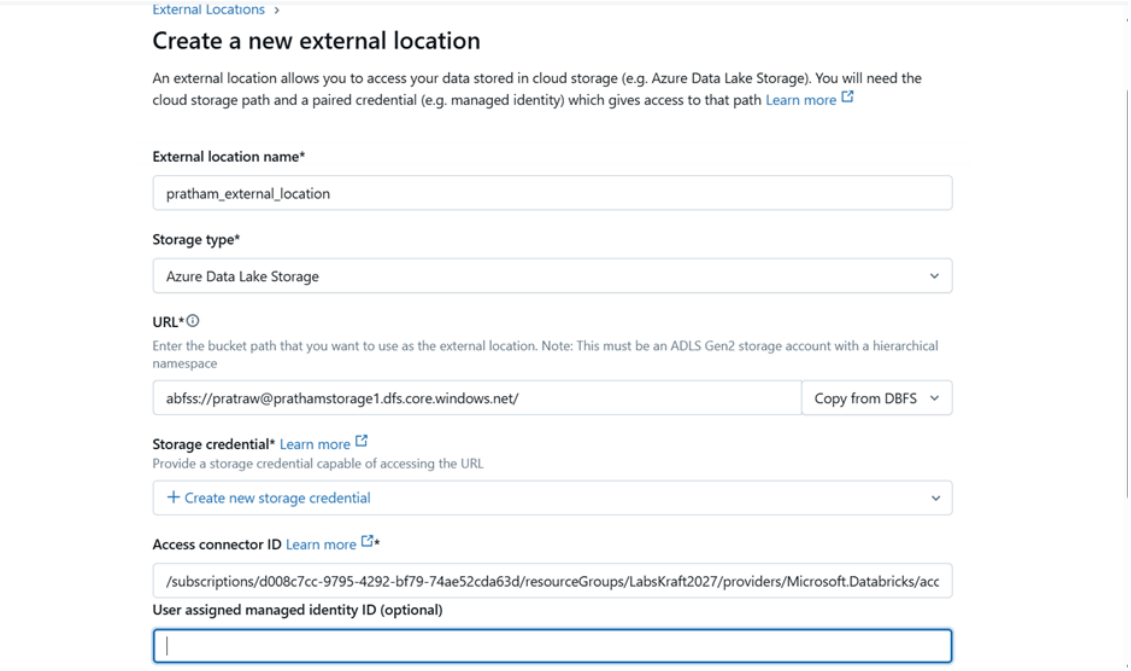
Creation of Access Connector:



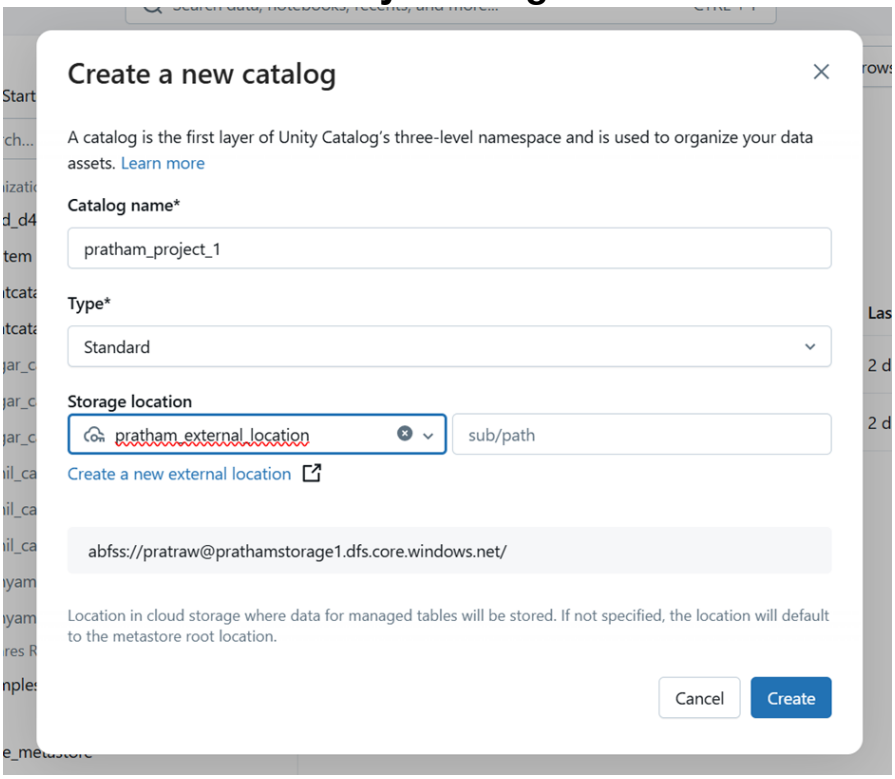
Adding Role Assignment:



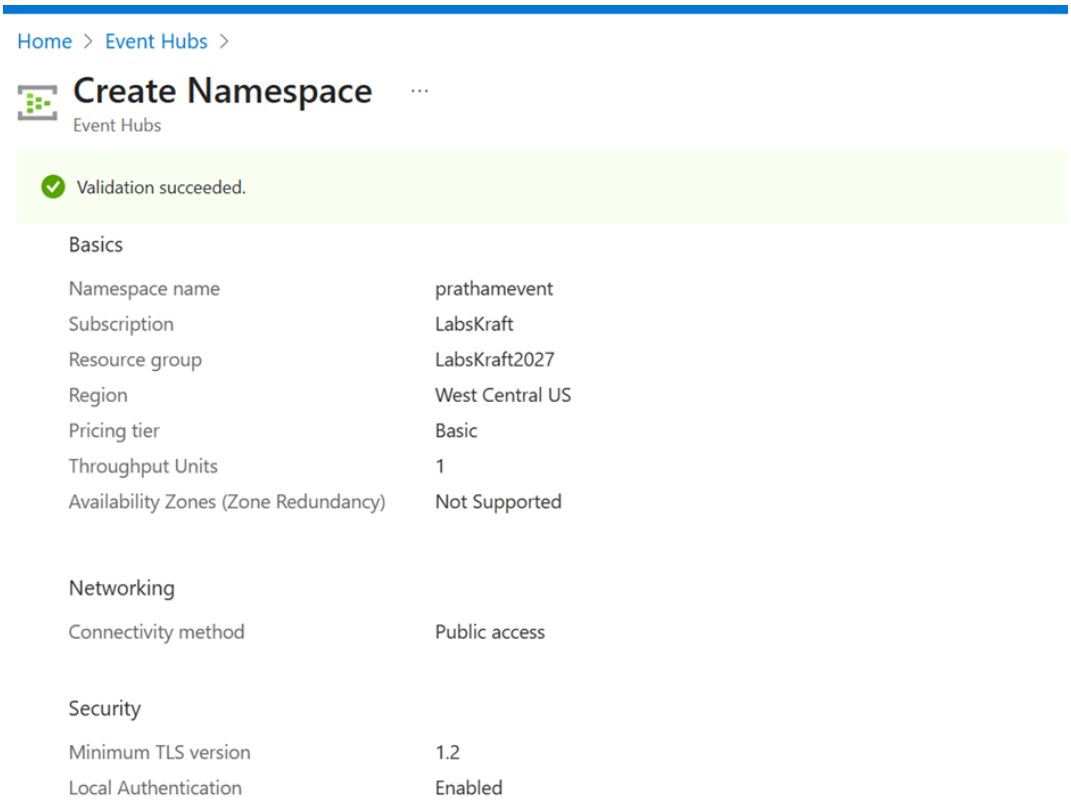
Creation Of External Location:

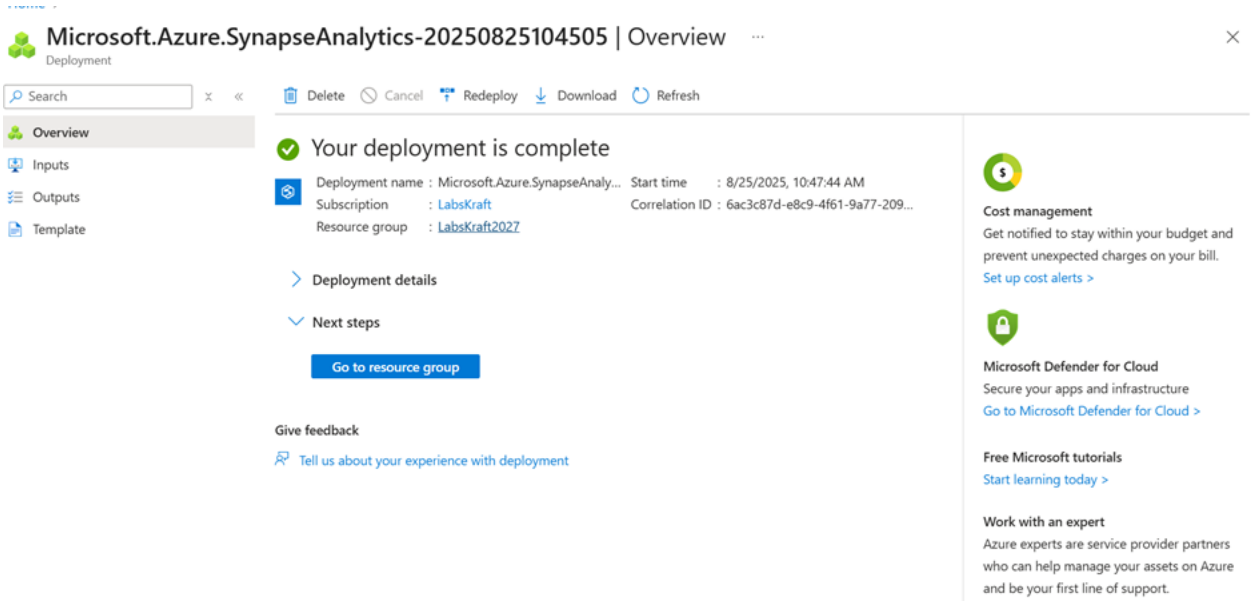


Creation Of New Unity Catalog:

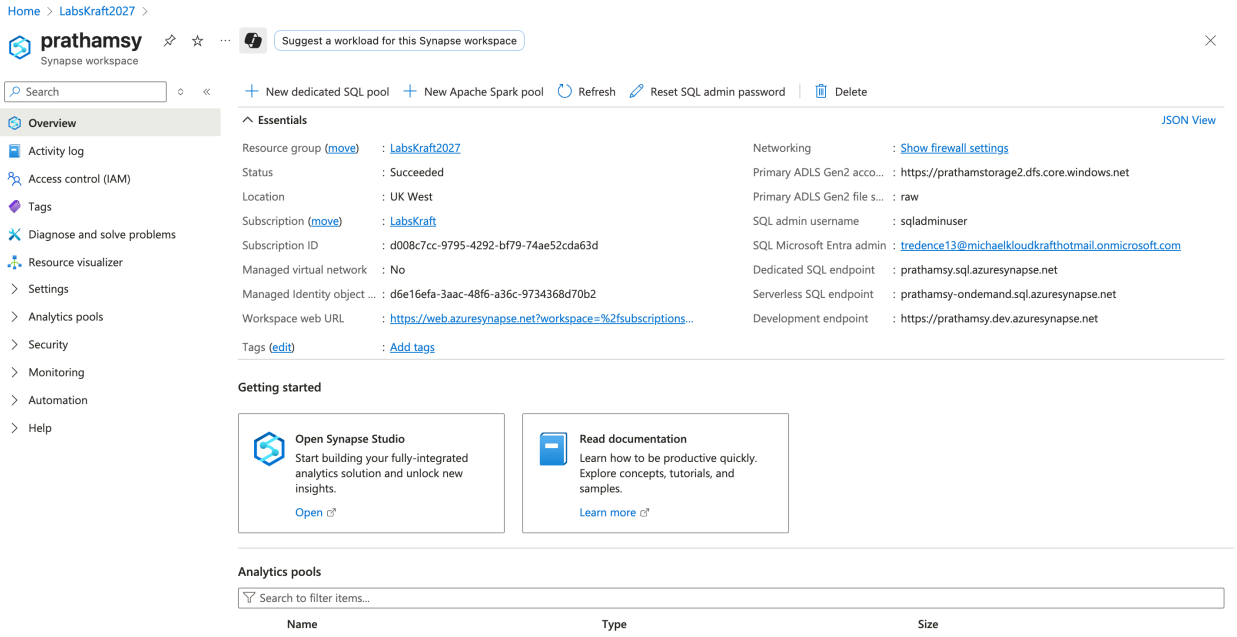


Creation of namespace & Eventhub:

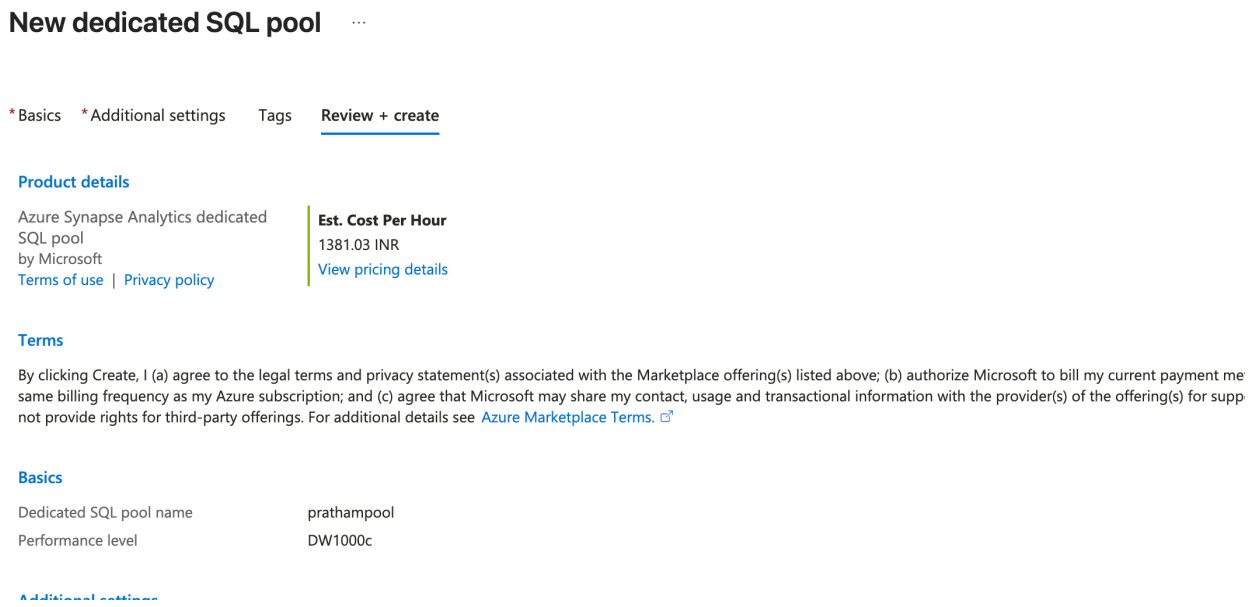




Creation of Synapse:



New SQL Pool:

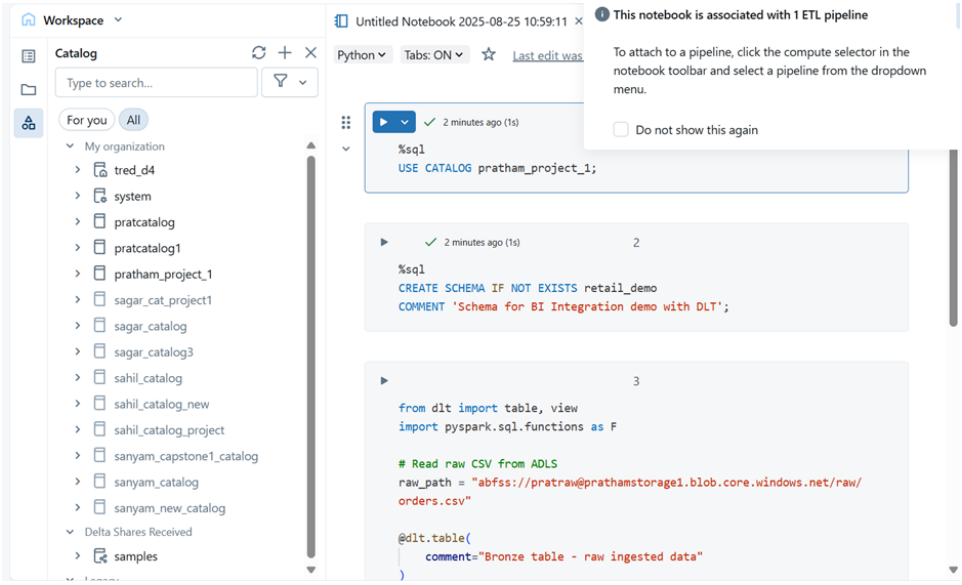


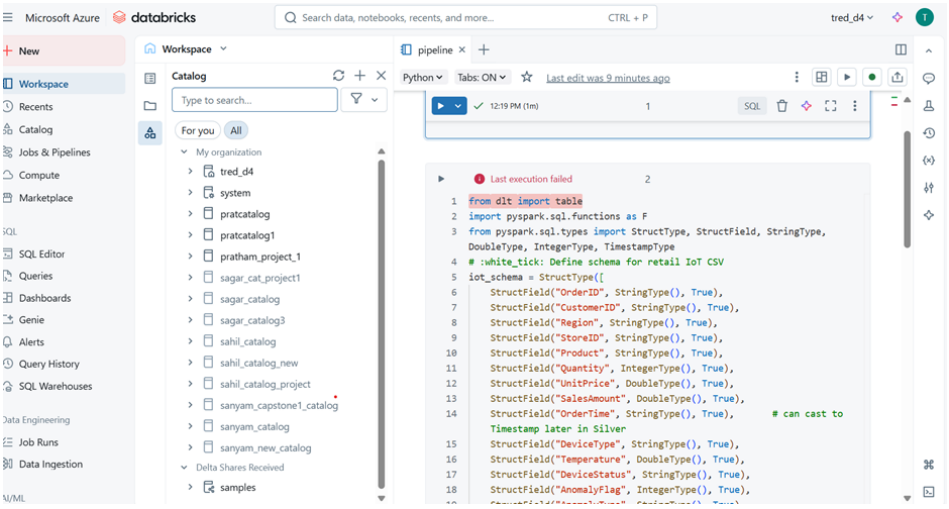
ADLS:

Bronze Layer:

```
from dlt import table
import pyspark.sql.functions as F
from pyspark.sql.types import StructType, StructField, StringType,
DoubleType, IntegerType, TimestampType
# :white_tick: Define schema for retail IoT CSV
iot_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Region", StringType(), True),
    StructField("StoreID", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),       # can cast to
Timestamp later in Silver
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])
# :white_tick: Paths (use subdirectories to avoid overlap with DLT
system files)
raw_path = "abfss://pratraw@prathamstorage1.dfs.core.windows.net/
raw_data/"
schema_path = "abfss://
pratraw@prathamstorage1.dfs.core.windows.net/dlt_schemas/
bronze_orders_pratham"
checkpoint_path = "abfss://
pratraw@prathamstorage1.dfs.core.windows.net/dlt_checkpoints/
bronze_orders_pratham"
@table(
    name="pratham_project_1.capstone_retail_pratham.bronze_orders",
    comment="Bronze table - raw ingested IoT retail data via Auto
Loader"
```

```
)
def bronze_orders():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", True)
            .option("cloudFiles.schemaLocation", schema_path)
            .schema(iot_schema)   # :white_tick: manually set schema (no
infer needed)
            .load(raw_path)       # :white_tick: points to /raw_data/ folder
    )
```



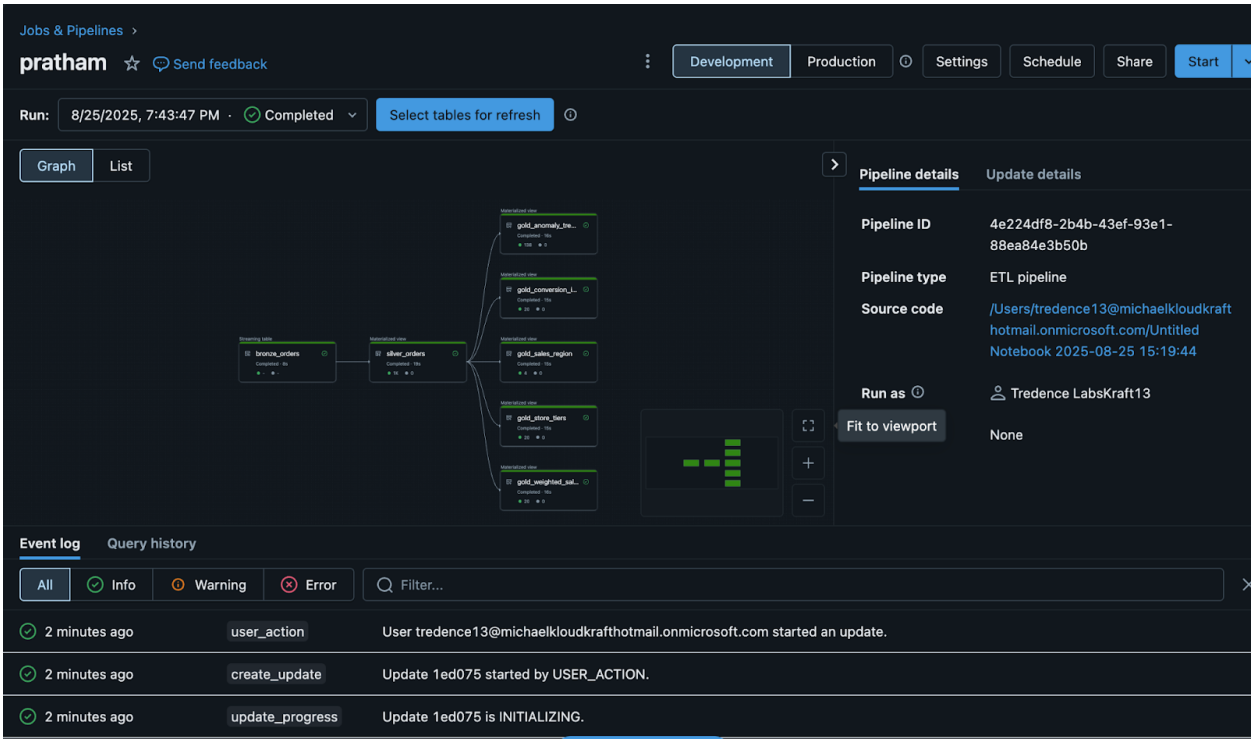


Combine of All three Layers:

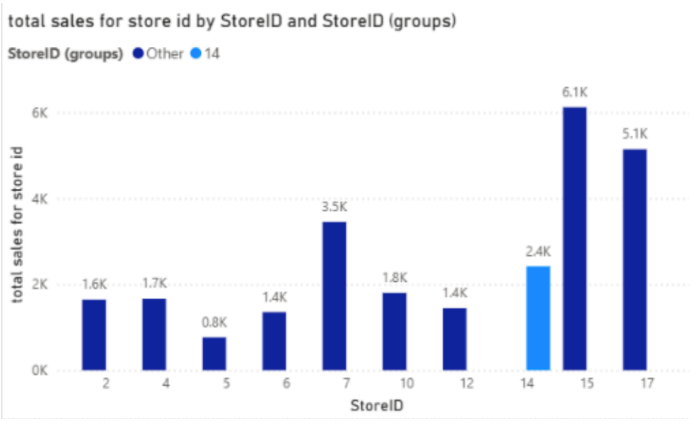
```
from dlt import table
import pyspark.sql.functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, IntegerType,
TimestampType
# :white_tick: Define schema for retail IoT CSV
iot_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Region", StringType(), True),
    StructField("StoreID", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),       # can cast to Timestamp later in Silver
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])
# :white_tick: Paths (use subdirectories to avoid overlap with DLT system files)
raw_path = "abfss://raw@prathamstorage2.dfs.core.windows.net/raw_data/"
schema_path = "abfss://raw@prathamstorage2.dfs.core.windows.net/dlt_schemas/
bronze_orders_pratham"
checkpoint_path = "abfss://raw@prathamstorage2.dfs.core.windows.net/dlt_checkpoints/
bronze_orders_pratham"
@table(
    name="pratham_02_project.capstone_retail_pratham.bronze_orders",
    comment="Bronze table - raw ingested IoT retail data via Auto Loader"
)
def bronze_orders():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", True)
            .option("cloudFiles.schemaLocation", schema_path)
            .schema(iot_schema)   # :white_tick: manually set schema (no infer needed)
            .load(raw_path)       # :white_tick: points to /raw_data/ folder
    )
# ========================================
# Silver Layer - Cleaned IoT Orders
# ========================================
@table(
    name="capstone_retail_pratham.silver_orders",
    comment="Silver layer: cleaned and deduplicated IoT orders with anomaly handling"
)
def silver_orders():
    # Read Bronze
    orders = dlt.read("pratham_02_project.capstone_retail_pratham.bronze_orders")
```

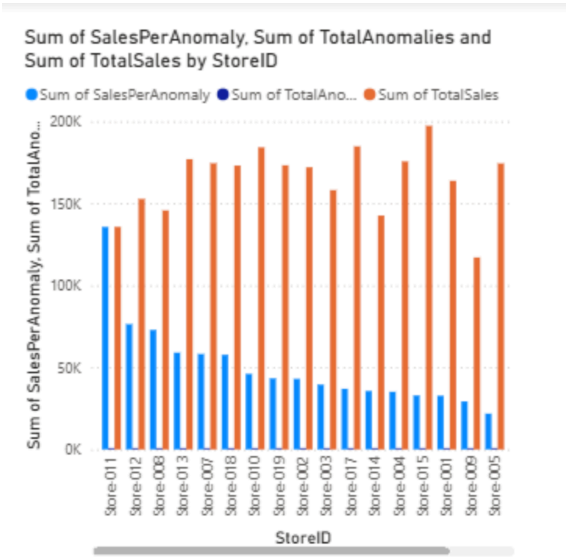
```
    # Standardize schema
    orders = (
        orders.withColumn("Quantity", F.col("Quantity").cast(IntegerType()))
              .withColumn("UnitPrice", F.col("UnitPrice").cast(DoubleType()))
              .withColumn("SalesAmount", F.col("SalesAmount").cast(DoubleType()))
              .withColumn("Temperature", F.col("Temperature").cast(DoubleType()))
              .withColumn("OrderTime", F.to_timestamp("OrderTime", "M/d/yyyy H:mm"))
    )
    # Deduplicate by OrderID
    orders = orders.dropDuplicates(["OrderID"])
    # Handle missing anomaly type
    orders = orders.withColumn(
        "AnomalyType",
        F.when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType").isNull()), "Unknown")
         .when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType") == ""), "Unknown")
         .otherwise(F.col("AnomalyType"))
    )
    # Add business rule fields
    orders = (
        orders.withColumn("IsAnomaly", F.col("AnomalyFlag") == 1)
              .withColumn("DeviceStatus", F.upper(F.col("DeviceStatus")))
              .withColumn("NormalizedStoreID", F.upper(F.col("StoreID")))
              .withColumn("NormalizedProduct", F.upper(F.col("Product")))
    )
    return orders

# ================================
# Gold Layer Tables (capstone_retail.gold)
# ================================
# 1. Daily / Monthly Sales per Region
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_sales_region",
    comment="Daily & Monthly Sales per Region"
)
def gold_sales_region():
    return (
        dlt.read("pratham_02_project.capstone_retail_pratham.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .withColumn("YearMonth", F.date_format("OrderDate", "yyyy-MM"))
        .groupBy("Region", "OrderDate", "YearMonth")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
# 2. Device Anomaly Trend per Store
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_anomaly_trends",
    comment="Device Anomaly Trend per Store"
)
def gold_anomaly_trends():
    return (
        dlt.read("pratham_02_project.capstone_retail_pratham.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .groupBy("StoreID", "OrderDate", "DeviceType", "AnomalyType")
        .agg(F.count("*").alias("AnomalyCount"))
    )
```

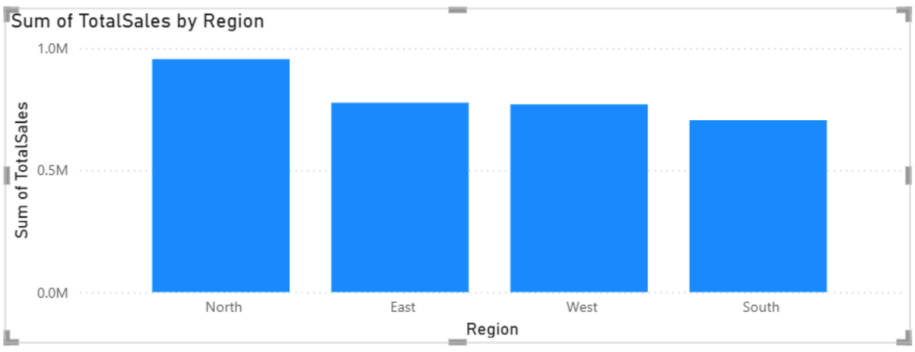
```
# 3. Conversion Rate Impact when devices fail
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_conversion_impact",
    comment="Conversion rate impact of device failures"
)
def gold_conversion_impact():
    df = dlt.read("pratham_02_project.capstone_retail_pratham.silver_orders")
    total_orders = df.groupBy("StoreID").agg(F.count("*").alias("TotalOrders"))
    failed_orders = (
        df.filter(df.AnomalyFlag == 1)
          .groupBy("StoreID")
          .agg(F.count("*").alias("FailedOrders"))
    )
    return (
        total_orders.join(failed_orders, "StoreID", "left")
        .withColumn("FailedOrders", F.coalesce(F.col("FailedOrders"), F.lit(0)))
        .withColumn("FailureRate", F.col("FailedOrders") / F.col("TotalOrders"))
    )
# 4. Store Tier Classification
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_store_tiers",
    comment="Store tiers based on total sales (High / Medium / Low performers)"
)
def gold_store_tiers():
    df = (
        dlt.read("pratham_02_project.capstone_retail_pratham.silver_orders")
        .groupBy("StoreID")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
    return (
        df.withColumn(
            "StoreTier",
            F.when(F.col("TotalSales") > 100000, "High")
             .when(F.col("TotalSales") > 50000, "Medium")
             .otherwise("Low")
        )
    )
# 5. Weighted Average Sales vs Anomalies
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_weighted_sales_anomalies",
    comment="Weighted average of sales vs anomalies per store"
)
def gold_weighted_sales_anomalies():
    df = dlt.read("pratham_02_project.capstone_retail_pratham.silver_orders")
    return (
        df.groupBy("StoreID")
        .agg(
            F.sum("SalesAmount").alias("TotalSales"),
            F.sum(F.when(F.col("AnomalyFlag") == 1, 1).otherwise(0)).alias("TotalAnomalies")
        )
        .withColumn("SalesPerAnomaly",
            F.when(F.col("TotalAnomalies") > 0, F.col("TotalSales") / F.col("TotalAnomalies"))
             .otherwise(F.lit(None))
        )
```



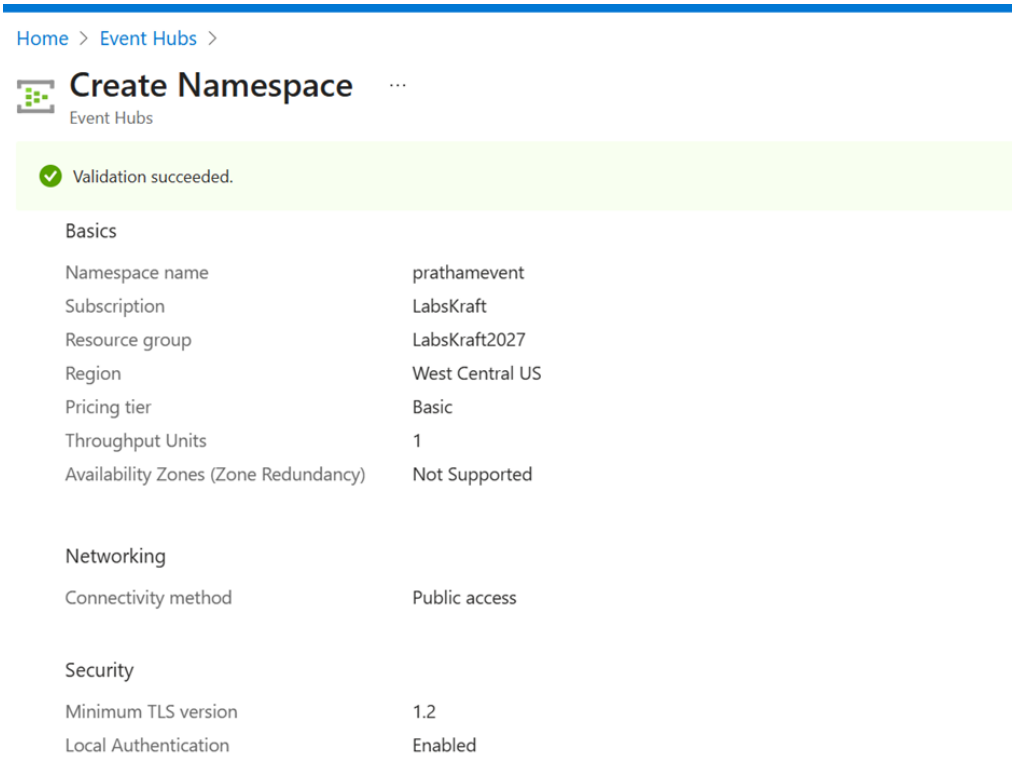
Power BI Outputs:



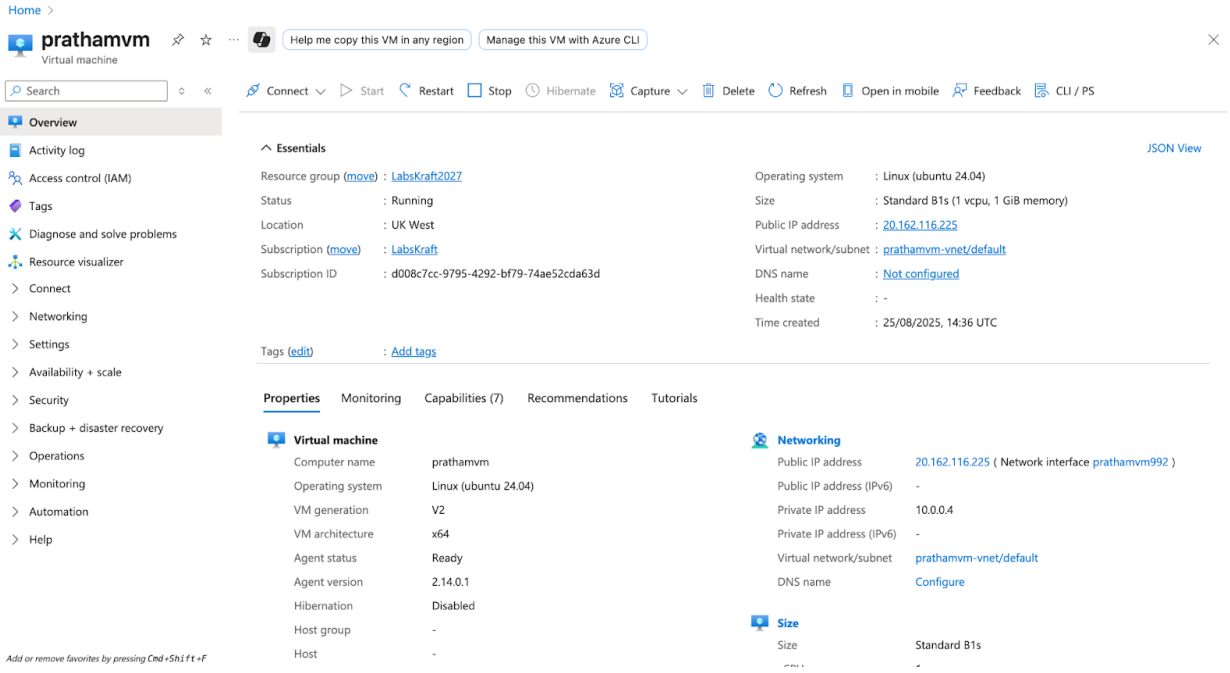




Creation of Event Namespace & EventHub:

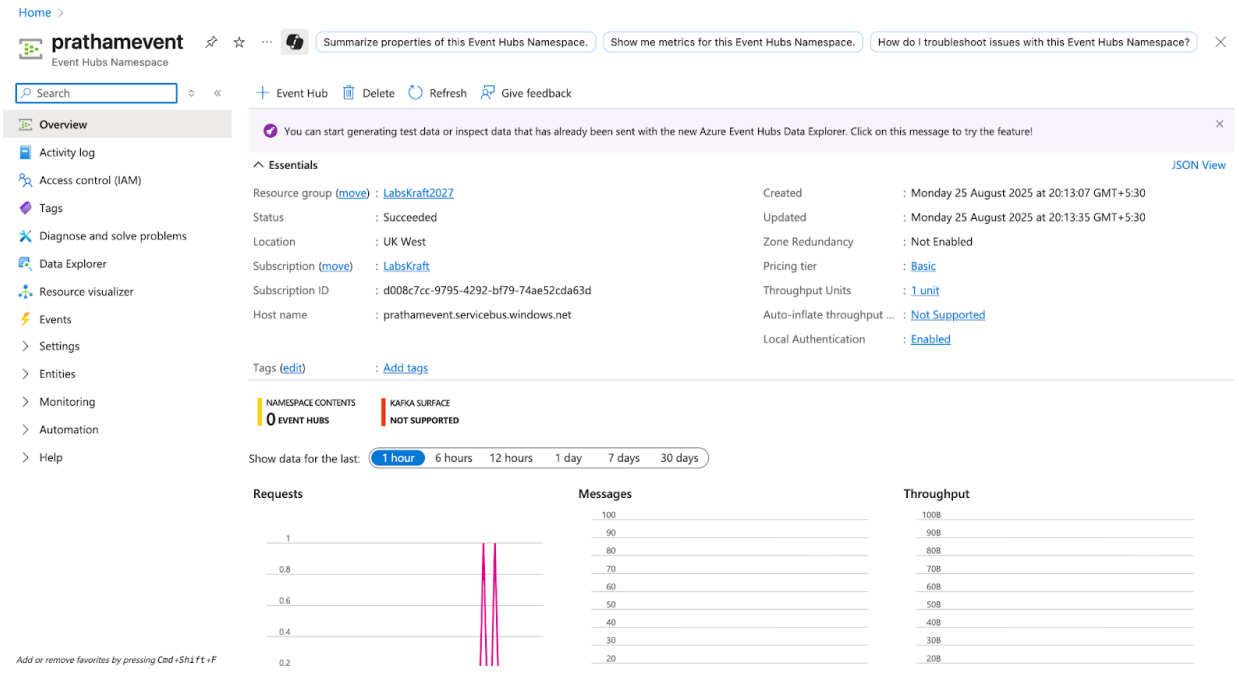


Creation Of Virtual Machine:



Code of GNU nano:

```
 GNU nano 7.2                                                    iot_stream.py
import random
import time
import uuid
from datetime import datetime
from azure.eventhub import EventHubProducerClient, EventData
# ------------------------------
# CONFIGURATION
# ------------------------------
EVENT_HUB_CONNECTION_STR = "Endpoint=sb://prathamevent.servicebus.windows.net/;SharedAccessKeyName=pratham111;SharedAccessKey=rQdXPcNn0JbKWtiRPYpAMoIPxi8luRTEh+AEhJr34+c=b"
EVENT_HUB_NAME = "prathameventb"
# ------------------------------
# Function to generate one IoT row
# ------------------------------
def generate_row():
    quantity = random.randint(1, 10)
    unit_price = round(random.uniform(50, 500), 2)
    sales_amount = round(quantity * unit_price, 2)
    device_status = random.choice(["Active", "Inactive", "Error"])
    anomaly_flag = 1 if device_status != "Active" else 0
    anomaly_type = random.choice(["Overheat","LowBattery","Disconnected"]) if anomaly_flag else "None"
    return {
        "OrderID": str(uuid.uuid4()),
        "CustomerID": f"CUST{random.randint(1000,9999)}",
        "Region": random.choice(["North","South","East","West"]),
        "StoreID": random.choice(["Store001","Store002","Store003"]),
        "Product": random.choice(["Laptop","Phone","Monitor","Keyboard","Mouse"]),
        "Quantity": quantity,
        "UnitPrice": unit_price,
        "SalesAmount": sales_amount,
        "OrderTime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "DeviceType": random.choice(["SensorA","SensorB","SensorC"]),
        "Temperature": round(random.uniform(20, 100), 2),
        "DeviceStatus": device_status,
        "AnomalyFlag": anomaly_flag,
        "AnomalyType": anomaly_type
    }
# ------------------------------
# Event Hub Producer
# ------------------------------
producer = EventHubProducerClient.from_connection_string(
    conn_str=EVENT_HUB_CONNECTION_STR,
    eventhub_name=EVENT_HUB_NAME
)
# ------------------------------
# Send data every second
# ------------------------------
try:
    for _ in range(1000): # Send exactly 1000 rows
        row = generate_row()
        event_data = EventData(str(row)) # convert dict to string
        with producer:
            producer.send_batch([event_data])
        print(f"Sent row: {row}")
        time.sleep(1) # 1-second interval
except KeyboardInterrupt:
    print("Streaming stopped by user.")
finally:
    producer.close()
```



Code of EventHub Config:

nano iot_stream.py

```
import random
import time
import uuid
from datetime import datetime
from azure.eventhub import EventHubProducerClient, EventData

# ------------------------------
# CONFIGURATION
# ------------------------------
Event connection string
EVENT_HUB_NAME = "prathamhub"

# ------------------------------
# Function to generate one IoT row
# ------------------------------
def generate_row():
  quantity = random.randint(1, 10)
  unit_price = round(random.uniform(50, 500), 2)
  sales_amount = round(quantity * unit_price, 2)
  device_status = random.choice(["Active", "Inactive", "Error"])
  anomaly_flag = 1 if device_status != "Active" else 0
  anomaly_type = random.choice(["Overheat","LowBattery","Disconnected"]) if anomaly_flag
else "None"

  return {
    "OrderID": str(uuid.uuid4()),
    "CustomerID": f"CUST{random.randint(1000,9999)}",
    "Region": random.choice(["North","South","East","West"]),
    "StoreID": random.choice(["Store001","Store002","Store003"]),
    "Product": random.choice(["Laptop","Phone","Monitor","Keyboard","Mouse"]),
    "Quantity": quantity,
    "UnitPrice": unit_price,
    "SalesAmount": sales_amount,
    "OrderTime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    "DeviceType": random.choice(["SensorA","SensorB","SensorC"]),
    "Temperature": round(random.uniform(20, 100), 2),
    "DeviceStatus": device_status,
    "AnomalyFlag": anomaly_flag,
    "AnomalyType": anomaly_type
  }

# ------------------------------
# Event Hub Producer
# ------------------------------
producer = EventHubProducerClient.from_connection_string(
  conn_str=EVENT_HUB_CONNECTION_STR,
  eventhub_name=EVENT_HUB_NAME
```

```
)

# ----------------------------
# Send data every second
# ----------------------------
try:
  for _ in range(1000): # Send exactly 1000 rows
    row = generate_row()
    event_data = EventData(str(row)) # convert dict to string
    with producer:
      producer.send_batch([event_data])
    print(f"Sent row: {row}")
    time.sleep(1) # 1-second interval
except KeyboardInterrupt:
  print("Streaming stopped by user.")
finally:
  producer.close()
```

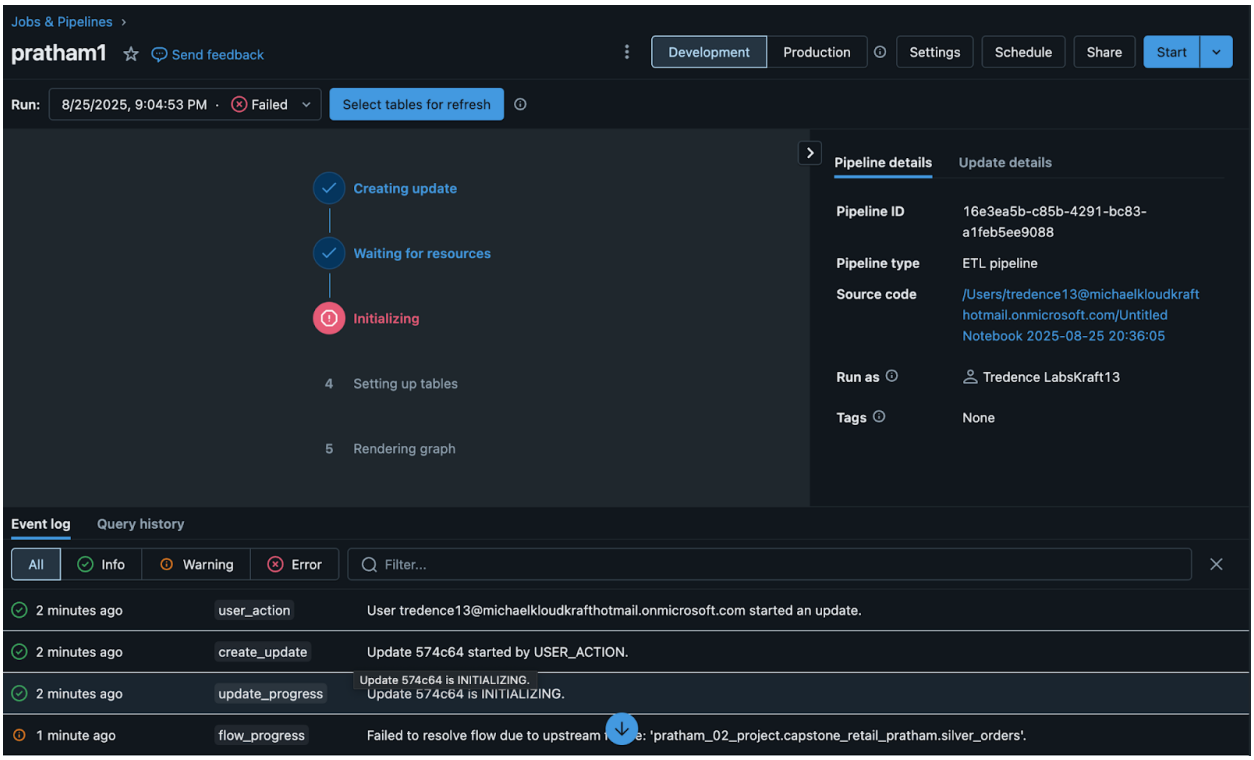
Code :

```
# ========================================
# Bronze → Silver → Gold with Event Hub Integration
# ========================================
from dlt import table, read
import pyspark.sql.functions as F
from pyspark.sql.types import *
# ================================
# :one: Bronze Layer: Schema & CSV
# ================================
iot_schema = StructType([
  StructField("OrderID", StringType(), True),
  StructField("CustomerID", StringType(), True),
  StructField("Region", StringType(), True),
  StructField("StoreID", StringType(), True),
  StructField("Product", StringType(), True),
  StructField("Quantity", IntegerType(), True),
  StructField("UnitPrice", DoubleType(), True),
  StructField("SalesAmount", DoubleType(), True),
  StructField("OrderTime", StringType(), True),
  StructField("DeviceType", StringType(), True),
  StructField("Temperature", DoubleType(), True),
  StructField("DeviceStatus", StringType(), True),
  StructField("AnomalyFlag", IntegerType(), True),
  StructField("AnomalyType", StringType(), True)
])
raw_path = "abfss://raw@prathamstorage2.dfs.core.windows.net/incoming/"
schema_path = "abfss://raw@prathamstorage2.dfs.core.windows.net/dlt_schemas/
bronze_orders_pratham"
@table(
  name="pratham_02_project.bronze.bronze_orders_csv",
```

```
  comment="Bronze table - raw IoT CSV"
)
def bronze_orders_csv():
   return (
    spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "csv")
      .option("header", True)
      .option("cloudFiles.schemaLocation", schema_path)
      .schema(iot_schema)
      .load(raw_path)
      .withColumn("OrderTime", F.to_timestamp("OrderTime"))
  )
# ================================
# :two: Bronze Layer: Event Hub IoT
# ================================
eventhub_connection_string = "Endpoint=sb://
prathamevent.servicebus.windows.net/;SharedAccessKeyName=manage;SharedAccessKey=F
KKqGhzAyDEqYXgNZvxzpYat2gUjW1VTS+AEhAIkesY=;EntityPath=event-vm"
consumer_group = "$Default"
ehConf = {
   'eventhubs.connectionString': eventhub_connection_string,
   'eventhubs.consumerGroup': consumer_group,
   'eventhubs.startingPosition': '@latest'
}
@table(
  name="pratham_02_project.bronze.bronze_orders_eventhub",
  comment="Bronze table - live IoT data from Event Hub"
)
def bronze_orders_eventhub():
  df = spark.readStream \
    .format("eventhubs") \
    .options(**ehConf) \
    .load()
  # Event Hub body is bytes → convert to string → parse JSON
  df1 = df.selectExpr("cast(body as string) as json_string")
  df_parsed = df1.select(F.from_json(F.col("json_string"),
iot_schema).alias("data")).select("data.*")
  return df_parsed.withColumn("OrderTime", F.to_timestamp("OrderTime"))
# ================================
# :three: Unified Bronze Table
# ================================
@table(
  name="pratham_02_project.capstone_retail_pratham.bronze_orders",
  comment="Unified Bronze table (CSV + Event Hub)"
)
def bronze_orders():
  df_csv = read("pratham_02_project.capstone_retail_pratham.bronze_orders_csv")
  df_eh = read("pratham_02_project.capstone_retail_pratham.bronze_orders_eventhub")
  return df_csv.unionByName(df_eh)
# ================================
# :four: Silver Layer
# ================================
```

```
@table(
  name="pratham_02_project.capstone_retail_pratham.silver_orders",
  comment="Silver layer: cleaned and deduplicated IoT orders with anomaly handling"
)
def silver_orders():
  orders = read("pratham_02_project.capstone_retail_pratham.bronze_orders")
  orders = (
    orders.withColumn("Quantity", F.col("Quantity").cast(IntegerType()))
      .withColumn("UnitPrice", F.col("UnitPrice").cast(DoubleType()))
      .withColumn("SalesAmount", F.col("SalesAmount").cast(DoubleType()))
      .withColumn("Temperature", F.col("Temperature").cast(DoubleType()))
      .withColumn("OrderTime", F.to_timestamp("OrderTime", "M/d/yyyy H:mm"))
  )
  orders = orders.dropDuplicates(["OrderID"])
  orders = orders.withColumn(
     "AnomalyType",
    F.when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType").isNull()), "Unknown")
     .when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType") == ""), "Unknown")
     .otherwise(F.col("AnomalyType"))
  )
  orders = (
    orders.withColumn("IsAnomaly", F.col("AnomalyFlag") == 1)
      .withColumn("DeviceStatus", F.upper(F.col("DeviceStatus")))
      .withColumn("NormalizedStoreID", F.upper(F.col("StoreID")))
      .withColumn("NormalizedProduct", F.upper(F.col("Product")))
  )
  return orders
# ================================
# :five: Gold Layer
# ================================
@table(
  name="pratham_02_project.capstone_retail_pratham.gold_sales_region",
  comment="Daily & Monthly Sales per Region"
)
def gold_sales_region():
   return (
    read("pratham_02_project.capstone_retail_pratham.silver_orders")
    .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
    .withColumn("YearMonth", F.date_format("OrderDate", "yyyy-MM"))
    .groupBy("Region", "OrderDate", "YearMonth")
    .agg(F.sum("SalesAmount").alias("TotalSales"))
  )
@table(
  name="pratham_02_project.capstone_retail_pratham.gold_anomaly_trends",
  comment="Device Anomaly Trend per Store"
)
def gold_anomaly_trends():
   return (
    read("pratham_02_project.capstone_retail_pratham.silver_orders")
    .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
    .groupBy("StoreID", "OrderDate", "DeviceType", "AnomalyType")
    .agg(F.count("*").alias("AnomalyCount"))
  )
@table(
```

```
  name="pratham_02_project.capstone_retail_pratham.gold_conversion_impact",
  comment="Conversion rate impact of device failures"
)
def gold_conversion_impact():
  df = read("pratham_02_project.capstone_retail_pratham.silver_orders")
  total_orders = df.groupBy("StoreID").agg(F.count("*").alias("TotalOrders"))
  failed_orders = (
    df.filter(df.AnomalyFlag == 1)
     .groupBy("StoreID")
     .agg(F.count("*").alias("FailedOrders"))
  )
  return (
    total_orders.join(failed_orders, "StoreID", "left")
    .withColumn("FailedOrders", F.coalesce(F.col("FailedOrders"), F.lit(0)))
    .withColumn("FailureRate", F.col("FailedOrders") / F.col("TotalOrders"))
  )
@table(
  name="pratham_02_project.capstone_retail_pratham.gold_store_tiers",
  comment="Store tiers based on total sales (High / Medium / Low performers)"
)
def gold_store_tiers():
  df = (
    read("pratham_02_project.capstone_retail_pratham.silver_orders")
    .groupBy("StoreID")
    .agg(F.sum("SalesAmount").alias("TotalSales"))
  )
  return (
    df.withColumn(
       "StoreTier",
      F.when(F.col("TotalSales") > 100000, "High")
       .when(F.col("TotalSales") > 50000, "Medium")
       .otherwise("Low")
    )
  )
@table(
  name="pratham_02_project.capstone_retail_pratham.gold_weighted_sales_anomalies",
  comment="Weighted average of sales vs anomalies per store"
)
def gold_weighted_sales_anomalies():
  df = read("pratham_02_project.capstone_retail_pratham.silver_orders")
  return (
    df.groupBy("StoreID")
    .agg(
      F.sum("SalesAmount").alias("TotalSales"),
      F.sum(F.when(F.col("AnomalyFlag") == 1, 1).otherwise(0)).alias("TotalAnomalies")
    )
    .withColumn("SalesPerAnomaly",
      F.when(F.col("TotalAnomalies") > 0, F.col("TotalSales") / F.col("TotalAnomalies"))
       .otherwise(F.lit(None))
    )
  )
```



Getting the Above Error.